REPORT :

SMART TRAFFIC DECISION SYSTEM

SUBMITTED BY : RIYAKRISHNA

25BHI10012

Fundamentals in AI ML

INTRODUCTION

If you've spent any time driving in the city lately, you know how frustrating traffic jams have become. There are more cars on the road than ever, they don't care if only one car is waiting or if dozens are backed up. It's no surprise you end up stuck at red lights with no one around, while a block over, drivers are piled up and barely moving. It wastes your time, burns more fuel, adds to air pollution, and honestly just makes everyone a little grumpier . We decided to try out a smarter approach. So, we built a ***Smart Traffic Decision*** System. Instead of forcing roads to wait their turn, it checks how many vehicles are actually waiting at every intersection, then gives more green light time to roads where traffic's heaviest. This dynamic timing helps decongest busy streets and keeps traffic moving. What's cool about this project is how it shows that even the basics of programming can help with big, real-world headaches. It also highlights how intelligent systems are starting to shape the future of city life, making infrastructure run smoother

Project Overview

Smart Traffic Decision System works: It's a Python simulation that models an intersection with several roads, each with a different number of cars. The system keeps an eye on traffic conditions and, using straightforward logic, spots which road is jammed up the most. When it sees a road with high traffic, it gives that lane extra green light time so vehicles can clear out faster. After each signal cycle, it checks again and updates vehicle counts, repeating the process just like a live traffic controller would.

We focused on making this system easy to build and understand. Everything relies on simple programming — if you can code loops, conditionals, and basic algorithms, you can follow along. Even though this is only a simulation, it does a solid job showing how smarter traffic systems work and how they could actually help real cities run better.

PROBLEM STATEMENT

Traffic management in most urban areas relies on fixed-time traffic signals that do not adapt to real-time traffic conditions. This leads to inefficient traffic flow, where vehicles experience unnecessary delays at empty intersections while heavily congested roads remain crowded. Such inefficiencies result in increased fuel consumption, higher pollution levels, and overall inconvenience for commuters.

The problem is to design a system that can intelligently manage traffic signals by analysing the number of vehicles on each road and dynamically allocating green signal time. The system should prioritize roads with higher traffic density to reduce congestion and improve overall traffic efficiency.

OBJECTIVES

Objectives of the Project

The main objectives of the Smart Traffic Decision System are:

- To design a system that dynamically controls traffic signals based on real-time vehicle density
- To reduce traffic congestion by prioritizing roads with higher vehicle count
- To minimize waiting time at traffic signals
- To improve overall traffic flow efficiency at intersections
- To demonstrate the application of algorithms (greedy approach) in solving real-world problems
- To develop a simple and effective simulation using Python

Methodology

Methodology of the Smart Traffic Decision System

The Smart Traffic Decision System is designed as a simulation model that manages traffic signals based on the number of vehicles on each road. The methodology followed in this project is explained step by step below:

1. Data Representation

Each road at the intersection is represented using a data structure (such as a list of dictionaries), where:

- Each road has a name
- Each road has a corresponding vehicle count

This allows easy storage and manipulation of traffic data.

2. Traffic Input

The system either takes user input or predefined values for the number of vehicles on each road. This simulates real-time traffic conditions at an intersection.

3. Traffic Analysis

The system analyzes all roads by comparing their vehicle counts.
It identifies the road with the highest number of vehicles using a **greedy approach,** where the locally optimal choice is made at each step.

4. Decision Making

Based on the traffic density, the system assigns green signal time dynamically:

- High traffic ☐ Longer green signal
- Medium traffic ☐ Moderate green signal
- Low traffic ☐ Short green signal

This ensures that congested roads are cleared faster.

5. Signal Execution

The selected road is given a green signal, allowing vehicles to pass.
At the same time, other roads remain on red signal.

6. Updating Traffic

After the signal is given:

- The number of vehicles on the selected road is reduced
- Other roads may remain the same or increase (optional simulation enhancement)

7. Continuous Operation

The system repeats the entire process in cycles:

- Analyze traffic
- Make decision
- Assign signal
- Update traffic

This simulates a real-time adaptive traffic control system.

The methodology ensures efficient traffic handling by continuously adapting to changing traffic conditions.

Algorithm

Algorithm: Smart Traffic Decision System

Input: Number of vehicles on each road
Output: Selected road for green signal and assigned time

Step 1: Start

Step 2: Initialize a list of roads with their respective vehicle counts

Step 3: Display the current traffic status of all roads

Step 4: Compare the number of vehicles on each road

Step 5: Select the road with the maximum number of vehicles
(This follows a greedy approach)

Step 6: Assign green signal time based on traffic density:

- If vehicles > 20 ☐ assign 60 seconds
- Else if vehicles > 10 ☐ assign 40 seconds
- Else ☐ assign 20 seconds

Step 7: Allow vehicles to pass through the selected road and reduce its vehicle count

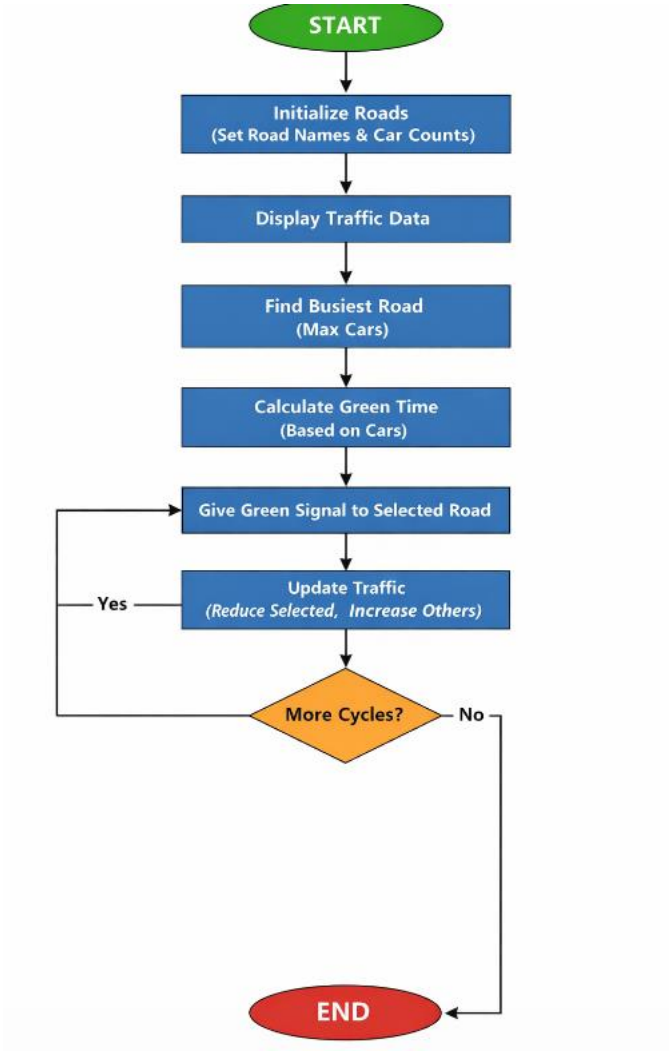
Step 8: Update traffic data for all roads

Step 9: Repeat Steps 3 to 8 for continuous traffic management

Step 10: Stop

The algorithm uses a greedy strategy by selecting the road with the highest traffic at each step to optimize traffic flow.

Flowchart


START
Initialize Roads
(Set Road Names & Car Counts)
Display Traffic Data
Find Busiest Road
(Max Cars)
Calculate Green Time
(Based on Cars)
Give Green Signal to Selected Road
Update Traffic
(Reduce Selected, Increase Others)
Yes
More Cycles?
No
END

Implementation

Implementation of Smart Traffic Decision System

The Smart Traffic Decision System is implemented using the Python programming language. The system simulates a traffic intersection with multiple roads and dynamically assigns signal timing based on vehicle density.

1. Programming Language Used

- Python is used due to its simplicity and ease of implementation
- Basic concepts such as lists, dictionaries, loops, and conditional statements are utilized

2. Data Structure Used

- A list of dictionaries is used to represent roads
- Each dictionary contains:
    - Road name
    - Number of vehicles

3. Core Functions Implemented

The system is divided into multiple functions for better structure and clarity:

a) calc_time()

- Determines signal duration based on vehicle count
- Uses conditional logic to assign time

b) show()

- Displays current traffic status of all roads
- Helps visualize system state at each step

c) find_busy()

- Identifies the road with the highest number of vehicles
- Implements a greedy approach

d) update()

- Reduces vehicle count after green signal
- Increases vehicles on other roads
- Simulates traffic flow

4. Working Process

1. The system initializes roads with vehicle counts
2. Traffic status is displayed
3. The road with maximum vehicles is selected
4. Green signal time is assigned dynamically
5. Vehicles are updated after the signal
6. The process repeats for multiple cycles

5. Output

The system displays:

- Current traffic status
- Selected road for green signal
- Time allocated for the signal

Time and Space Complexity

Time Complexity

The Smart Traffic Decision System processes all roads during each cycle to make decisions.

- Displaying traffic for all roads takes O(n) time
- Finding the road with maximum vehicles requires scanning all roads ☐ O(n)
- Updating vehicle counts also takes O(n)

Therefore, the total time complexity per cycle is:

O(n), where *n* is the number of roads

This means the system scales linearly with the number of roads.

The system stores information about each road and its vehicle count using a list of dictionaries.

- Storage required is proportional to the number of roads

space complexity is:

O(n)

The algorithm is efficient for real-time applications as both time and space complexity grow linearly with the number of roads.

Screenshots and output

```
Cycle 1

Current traffic:
A has 23 cars
B has 8 cars
C has 14 cars
D has 5 cars
Green signal for road A
Time: 55 seconds
```

```
Cycle 5

Current traffic:
A has 9 cars
B has 7 cars
C has 13 cars
D has 17 cars
Green signal for road D
Time: 35 seconds

Simulation finished
```

```
Cycle 2

Current traffic:
A has 13 cars
B has 11 cars
C has 17 cars
D has 8 cars
Green signal for road C
Time: 35 seconds

Cycle 3

Current traffic:
A has 16 cars
B has 14 cars
C has 7 cars
D has 11 cars
Green signal for road A
Time: 35 seconds
```

Challenges Faced

Challenges in Developing the Smart Traffic Decision System

During the development of this project, several challenges were encountered:

1. Designing Fair Signal Logic

It was challenging to decide how much green signal time should be assigned for different traffic levels. The system needed to balance between clearing congestion and not starving other roads of signal time.

2. Choosing Appropriate Thresholds

Selecting suitable thresholds (e.g., 10 and 20 vehicles) for categorizing traffic into low, medium, and high levels required careful consideration to ensure realistic behavior.

3. Simulating Real-World Traffic

Real traffic conditions are dynamic and unpredictable. Simulating this behavior using simple code without real sensors or live data was a limitation and required assumptions.

4. Handling Continuous Updates

Ensuring that the system updates traffic conditions smoothly in each cycle without abrupt or unrealistic changes was another challenge.

5. Maintaining Simplicity and Clarity

While improving the system, it was important to keep the logic simple and understandable, especially for demonstration purposes.

These challenges helped in improving the design and provided a deeper understanding of real-world system behaviour.

Future Scope

Future Scope of Smart Traffic Decision System

The current system is a basic simulation, but it can be extended into a more advanced and practical solution. Some possible improvements include:

1. Integration with Real-Time Sensors

The system can be connected to real-world sensors or cameras to collect live traffic data instead of using predefined values. This would make the system more accurate and practical.

2. Use of Machine Learning

Machine learning algorithms can be applied to predict traffic patterns based on historical data. This would allow the system to make smarter decisions in advance.

3. Emergency Vehicle Detection

The system can be enhanced to detect emergency vehicles such as ambulances and fire trucks, giving them immediate priority at signals.

4. Pedestrian and Weather Considerations

Future versions can include pedestrian crossings and weather conditions (like rain or fog) to adjust signal timing more effectively.

5. Graphical User Interface (GUI)

A user-friendly interface can be developed to visualize traffic flow, signal changes, and system decisions in real time.

6. Smart City Integration

The system can be integrated into smart city infrastructure, where multiple intersections communicate and coordinate to optimize overall traffic flow.

With these enhancements, the system can evolve from a simple simulation into a fully intelligent traffic management solution.

REFERENCES

1. **Greedy Algorithms**
   https://www.geeksforgeeks.org/dsa/greedy-algorithms/

2. **Smart Traffic Systems**
   https://ieeexplore.ieee.org/document/5203289